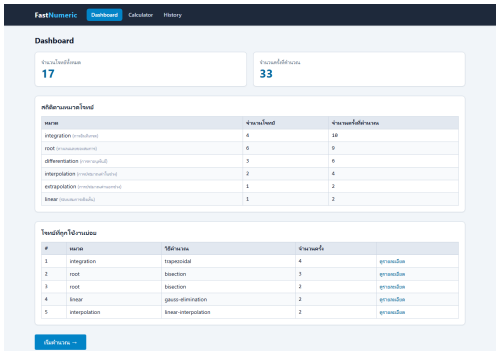
รายงานโครงงาน

FastNumeric

เว็บแอปพลิเคชันคำนวณวิธีเชิงตัวเลขพร้อมแสดงขั้นตอนการคำนวณ



จัดทำโดย

[ต้องเพิ่มเติมข้อมูล] รหัสนักศึกษา [ต้องเพิ่มเติมข้อมูล]

[ต้องเพิ่มเติมข้อมูล] รหัสนักศึกษา [ต้องเพิ่มเติมข้อมูล]

เสนอ

อาจารย์ [ต้องเพิ่มเติมข้อมูล]

รายงานนี้เป็นส่วนหนึ่งของรายวิชา [ต้องเพิ่มเติมข้อมูล]

ภาควิชา [ต้องเพิ่มเติมข้อมูล] คณะ [ต้องเพิ่มเติมข้อมูล]

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ภาคการศึกษาที่ [ต้องเพิ่มเติมข้อมูล] ปีการศึกษา [ต้องเพิ่มเติมข้อมูล]

บทที่ 1
บทนำ

1.1 ที่มาและความสำคัญของโปรเจ็ค

ในการเรียนวิชาวิธีเชิงตัวเลข (Numerical Methods) ผู้เรียนต้องคำนวณคำตอบของปัญหาทางคณิตศาสตร์หลายประเภท เช่น การหารากของสมการ การแก้ระบบสมการเชิงเส้น การแทรกและการคาดคะเนค่า ตลอดจนการหาพื้นที่ใต้เส้นโค้งและอนุพันธ์เชิงตัวเลข ซึ่งแต่ละวิธีต้องทำการคำนวณซ้ำ ๆ จำนวนมาก ถ้าทำด้วยมือจะเสียเวลามากและเกิดข้อผิดพลาดได้ง่าย นอกจากนี้ผู้เรียนยังควรเข้าใจ **ขั้นตอนการคำนวณ** ของแต่ละวิธี ไม่ใช่แค่คำตอบสุดท้ายเท่านั้น

โครงงาน **FastNumeric** จึงเป็นเว็บแอปพลิเคชันสำหรับคำนวณวิธีเชิงตัวเลขจำนวน 6 หมวด 11 วิธี โดยโปรแกรมไม่เพียงแสดงคำตอบสุดท้าย แต่แสดง **ขั้นตอนการคำนวณ** และ **ตารางการทำซ้ำ (Iteration Table)** ครบทุกขั้นตอน ผู้ใช้สามารถตรวจสอบการคำนวณย้อนกลับได้ทีละรอบ เหมาะกับการใช้ตรวจคำตอบและทบทวนวิธีทำของผู้เรียน และโปรแกรมยังบันทึกประวัติการคำนวณลงฐานข้อมูล นับจำนวนครั้งที่คำนวณโจทย์เดิม และสรุปสถิติบนหน้า Dashboard

ด้านการพัฒนา โปรแกรมถูกออกแบบด้วยหลัก**การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP)** อย่างเต็มรูปแบบ ทั้งการซ่อนรายละเอียด (Encapsulation) การสืบทอดคุณสมบัติ (Inheritance) ความเป็นนามธรรม (Abstraction) รูปแบบพฤติกรรมหลากหลาย (Polymorphism) และการประกอบวัตถุ (Composition) เพื่อให้ระบบแยกชั้นความรับผิดชอบชัดเจนระหว่างชั้นรับข้อมูล ชั้นอัลกอริทึม และชั้นแสดงผล/จัดเก็บ ทำให้เพิ่มวิธีคำนวณใหม่ได้โดยไม่ต้องแก้โค้ดเดิม

1.2 ประเภทของโครงการ

ตารางที่ 1.1 ประเภทของโครงการ ภาษา และเทคโนโลยีที่ใช้

หัวข้อ	รายละเอียด
ประเภทของโปรแกรม	เว็บแอปพลิเคชันประเภทพัฒนาซอฟต์แวร์ประยุกต์ (Application Development) ทำงานบนเว็บเบราว์เซอร์ ประกอบด้วยส่วนติดต่อผู้ใช้ (Frontend) ส่วนประมวลผล (Backend) และฐานข้อมูล
ภาษาที่ใช้	JavaScript (Node.js ฝั่งเซิร์ฟเวอร์ และ React ฝั่งผู้ใช้)
เทคโนโลยีที่ใช้	React 19 + Vite + Tailwind CSS (Frontend), Node.js + Express (REST API), SQLite ผ่าน node:sqlite (ฐานข้อมูล), Docker + Docker Compose (การติดตั้ง), Git (จัดการเวอร์ชัน)
ข้อจำกัดที่ตั้งใจ	ไม่ใช้ไลบรารีคำนวณสำเร็จรูปสำหรับอัลกอริทึมทั้ง 11 วิธี เขียนเองทั้งหมด และไม่ใช้ eval() ในการแปลงสมการ f(x) เพื่อความปลอดภัย

1.3 ประโยชน์

1. คำนวณวิธีเชิงตัวเลขได้ถูกต้อง รวดเร็ว ครอบคลุม 6 หมวด 11 วิธีที่ใช้เรียนในรายวิชาวิธีเชิงตัวเลข
2. แสดงขั้นตอนการคำนวณและตารางการทำซ้ำทุกวิธี ช่วยให้ผู้เรียนเข้าใจกลไกของอัลกอริทึม ไม่ใช่แค่คำตอบ
3. ตรวจคำตอบการบ้านหรือการคำนวณด้วยมือได้รวดเร็ว ลดข้อผิดพลาดจากการคำนวณซ้ำ ๆ

4. บันทึกประวัติการคำนวณลงฐานข้อมูล นับจำนวนครั้งที่คำนวณโจทย์เดิม และสรุปสถิติให้เห็นภาพรวม
5. เป็นตัวอย่างการนำหลัก OOP (Encapsulation, Inheritance, Abstraction, Polymorphism, Composition) ไปใช้กับโจทย์จริงด้านคณิตศาสตร์ เช่น การแยกชั้นอัลกอริทึมออกจากการแสดงผล

1.4 ขอบเขตของโครงการ

ขอบเขตของโครงการครอบคลุมการคำนวณวิธีเชิงตัวเลข 6 หมวด 11 วิธี พร้อมระบบประวัติการคำนวณและ Dashboard สรุปสถิติ โดยตรวจสอบกับซอร์สโค้ดจริงของโปรแกรม

1.4.1 ฟังก์ชันของโปรแกรม

ตารางที่ 1.2 ฟังก์ชันการคำนวณของแต่ละหมวด

หมวด	วิธีคำนวณ	คุณสมบัติที่พัฒนา
Root of Equation	Bisection, Newton-Raphson	หารากของสมการ f(x) = 0 แสดงตารางการทำซ้ำ ตรวจเงื่อนไข f(XL)·f(XU) < 0 และเงื่อนไขความลู่เข้า
Linear Algebra	Gauss Elimination	แก้ระบบสมการเชิงเส้น n ตัวแปร (n = 2–6) ด้วยการตัดตัวแปรแบบ Partial Pivoting ตรวจเมทริกซ์เอกฐาน
Interpolation	Linear, Lagrange	หาค่าระหว่างจุดจากตาราง (x, y) ตรวจว่า x อยู่ในช่วงข้อมูลหรือไม่
Extrapolation	Linear	คาดคะเนค่านอกช่วงข้อมูล ตรวจว่า x ต้องอยู่นอกช่วงจริง
Integration	Trapezoidal, Simpson's 1/3	หาพื้นที่ใต้เส้นโค้งจากตารางจุด ตรวจว่า Simpson ต้องมีจำนวนช่วงเป็นเลขคู่ รองรับกรณี a > b
Differentiation	Forward, Backward, Central	หาอนุพันธ์เชิงตัวเลขจากตารางจุด ตรวจว่าจุดที่เลือกมีเพื่อนบ้านครบตามสูตร

1.4.2 คุณสมบัติเชิงระบบ

- กรอกสมการ f(x) ได้โดยตรง เช่น `x^3 - x - 2` โปรแกรมแยกวิเคราะห์สมการเองด้วย Expression Parser ที่เขียนขึ้นเอง (ไม่ใช้ eval)
- กำหนดค่า Tolerance และ Max Iterations ได้ พร้อมค่าเริ่มต้นที่เหมาะสม
- จัดการข้อผิดพลาดเป็นภาษาไทยที่อธิบายสาเหตุ เช่น หารด้วยศูนย์, เมทริกซ์เอกฐาน, ไม่ลู่เข้า
- บันทึกประวัติการคำนวณลง SQLite นับโจทย์ซ้ำ (หมวด + วิธี + อินพุตเดิม) ลบรายการได้
- หน้า Dashboard สรุปสถิติ: จำนวนโจทย์, จำนวนครั้งที่คำนวณ, สัดส่วนตามหมวด, โจทย์ยอดนิยม
- ติดตั้งด้วย Docker Compose พร้อม health check และ named volume สำหรับเก็บฐานข้อมูลถาวร

1.4.3 ตารางเวลาการดำเนินโครงการ

ตารางที่ 1.3 แผนการดำเนินโครงการ 6 สัปดาห์

กิจกรรม	สัปดาห์ 1	สัปดาห์ 2	สัปดาห์ 3	สัปดาห์ 4	สัปดาห์ 5	สัปดาห์ 6
ศึกษาโจทย์และวิธีเชิงตัวเลขทั้ง 11 วิธี	✓					
ออกแบบโครงสร้างคลาส (OOP) และ REST API		✓				
พัฒนาคลาสอัลกอริทึม + Expression Parser		✓	✓			
พัฒนาฐานข้อมูลและระบบประวัติการคำนวณ			✓	✓		

กิจกรรม	สัปดาห์ 1	สัปดาห์ 2	สัปดาห์ 3	สัปดาห์ 4	สัปดาห์ 5	สัปดาห์ 6
พัฒนาส่วนติดต่อผู้ใช้ (React + Tailwind)			✓	✓		
เขียนชุดทดสอบและตรวจสอบผลลัพธ์				✓	✓	
จัดทำ Docker / เอกสาร / รายงาน					✓	✓

บทที่ 2
การพัฒนา

2.1 ภาพรวมของระบบ

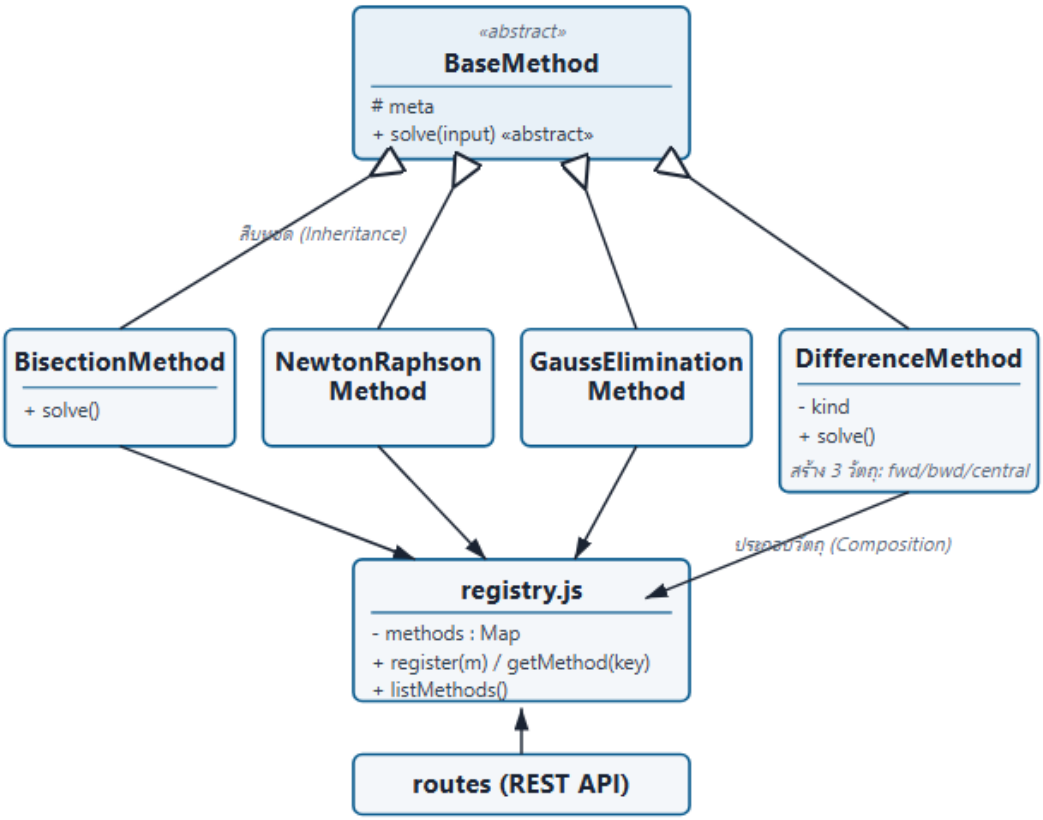
FastNumeric เป็นเว็บแอปพลิเคชันแบบ 3 ชั้น (Three-tier) ประกอบด้วย **ชั้นรับข้อมูล** (ฟอร์ม React สร้างจาก meta ของแต่ละวิธี), **ชั้นอัลกอริทึม** (คลาสวิธีคำนวณ 11 วิธีสืบทอดจากคลาสนามธรรม BaseMethod) และ **ชั้นแสดงผล/จัดเก็บ** (ResultView แสดงผลจากโครงสร้างผลลัพธ์มาตรฐาน, history.js บันทึกลง SQLite) การทำงานเริ่มจากผู้ใช้เลือกวิธีคำนวณและกรอกอินพุต ฝั่งเซิร์ฟเวอร์ตรวจอินพุตที่ด่าน validation แล้วเรียก `method.solve(input)` ผลลัพธ์ที่ได้มีโครงสร้างมาตรฐานเดียวกันทุกวิธี ได้แก่ `{ answer, steps, table, meta }` จึงแสดงผลด้วยหน้าจอเดียวกันได้ทั้ง 11 วิธี และบันทึกประวัติลงฐานข้อมูลโดยอัตโนมัติ

ตารางที่ 2.1 หน้าที่ของไฟล์หลักในแต่ละส่วนของระบบ

ไฟล์ / ส่วน	หน้าที่
algorithms/BaseMethod.js	คลาสนามธรรมของทุกวิธีคำนวณ เก็บ meta (key, หมวด, ชื่อ, นิยามฟิลด์อินพุต) และประกาศเมธอดนามธรรม solve() ให้คลาสลูก implement
algorithms/roots.js, linear.js, interpolation.js, extrapolation.js, integration.js, differentiation.js	คลาสวิธีคำนวณจริง 11 วิธี สืบทอดจาก BaseMethod แต่ละคลาสรับผิดชอบเฉพาะคณิตศาสตร์ของตัวเอง
algorithms/registry.js	ศูนย์กลางลงทะเบียนวัตถุวิธีคำนวณทั้งหมดใน Map พร้อมเมธอด listMethods() สำหรับสร้าง UI อัตโนมัติ
expression/parser.js	แปลงสมการ f(x) จากสตริงเป็นฟังก์ชันด้วย Tokenizer + Recursive Descent Parser (ไม่ใช้ eval)
validation/index.js	ด่านตรวจอินพุตกลาง (asFloat, asInt, asPoints, asMatrix, asExpression) โยน ValidationError พร้อมข้อความไทย
database/db.js + history.js	ชั้นเข้าถึงข้อมูล: ตาราง problem + calculation ใน SQLite นับโจทย์ซ้ำด้วย input hash
routes/index.js	REST API: GET /api/methods, POST /api/calculate, GET /api/history, GET /api/stats
client (React)	MethodInput สร้างฟอร์มอัตโนมัติจาก meta.fields, ResultView แสดงคำตอบ/ขั้นตอน/ตาราง

2.2 คลาสไดอะแกรม พร้อมคำอธิบาย

โครงสร้างคลาสหลักของระบบออกแบบตามหลักการแบ่งชั้น โดยคลาสนามธรรม **BaseMethod** เป็นแม่ของคลาสวิธีคำนวณทั้ง 11 วิธี และมี **registry.js** เป็นศูนย์กลางประกอบ (compose) วัตถุทุกวิธีเข้าด้วยกัน รูปที่ 2.1 แสดงความสัมพันธ์ของคลาสส่วนอัลกอริทึม สังเกตว่าวิธีอนุพันธ์ทั้งสามตัว (Forward / Backward / Central) มาจากคลาสเดียวกันคือ `DifferenceMethod` แต่ถูกสร้างเป็น 3 วัตถุด้วยค่า kind ต่างกัน



รูปที่ 2.1 Class Diagram ของระบบ FastNumeric (ส่วนอัลกอริทึม)

ความสัมพันธ์หลักในรูปที่ 2.1 ได้แก่ คลาสวิธีคำนวณทุกคลาส**สืบทอด** (Inheritance) จาก `BaseMethod` และ `registry.js` **ประกอบวัตถุ** (Composition) ของวิธีคำนวณทั้ง 11 วัตถุเข้าเป็นระบบเดียว ส่วน `routes` (REST API) เข้าถึงวิธีคำนวณผ่าน `getMethod(key)` เท่านั้น ไม่รู้จักคลาสใดโดยตรง ทำให้เพิ่มวิธีใหม่ได้โดยสร้างคลาสใหม่ `extends BaseMethod` แล้วเพิ่มบรรทัดเดียวใน registry.js

2.3 รูปแบบการพัฒนา Application

โครงงานนี้พัฒนาแบบเพิ่มทีละส่วน (Iterative and Incremental Development) เริ่มจากแกนหลักที่คำนวณได้จริงขนาดเล็ก แล้วเพิ่มวิธีคำนวณ ฐานข้อมูล และหน้าจอทีละชั้น โดยแต่ละรอบมีการวางแผน ออกแบบ เขียนโค้ด ทดสอบ และแก้ไข รายละเอียดแต่ละขั้นตอนมีดังนี้

1. **Planning** กำหนดขอบเขต 6 หมวด 11 วิธี และข้อจำกัด: เขียนอัลกอริทึมเองทั้งหมด ไม่ใช้ eval() และข้อความ error ต้องเป็นภาษาไทยที่อธิบายสาเหตุได้
2. **Analyze / Design** ออกแบบคลาสนามธรรม BaseMethod และโครงสร้างผลลัพธ์มาตรฐาน กำหนดกฎการพึ่งพาระหว่างชั้น (อัลกอริทึมไม่รู้จัก HTTP และฐานข้อมูล validation เป็นด่านเดียวของอินพุต)
3. **Implementation รอบที่ 1** คลาสอัลกอริทึมทั้ง 11 วิธี + Expression Parser + ระบบ validation
4. **Implementation รอบที่ 2** ฐานข้อมูล SQLite (ตาราง problem / calculation) และ REST API ทั้งหมด
5. **Implementation รอบที่ 3** หน้าเว็บ React: เลือกวิธี, ฟอร์มอัตโนมัติจาก meta, หน้าผลลัพธ์, ประวัติ, Dashboard

6. **Testing และแก้ไข** ทดสอบอัลกอริทึมด้วยโจทย์ที่รู้คำตอบจริง ทดสอบกรณี error และทดสอบผ่านหน้าเว็บจริง พบและแก้บั๊กระหว่าง Frontend กับ Backend อย่างน้อย 1 จุด (ดังรายละเอียดในบทที่ 3)

โปรแกรมเริ่มทำงานจากไฟล์ `server/src/index.js` ซึ่งสร้างแอป Express, เปิด CORS ตามค่า environment และต่อ Router หลักจาก `routes/index.js` ฝั่งหน้าเว็บสร้างด้วย Vite เมื่อติดตั้งด้วย Docker Compose จะ build หน้าเว็บเป็นไฟล์ static แล้วเสิร์ฟผ่าน Nginx พร้อม reverse proxy ไปยังเซิร์ฟเวอร์ Node.js

2.4 อธิบายส่วนของโปรแกรมที่มี (หลักการ OOP)

2.4.1 Constructor

คลาสวิธีคำนวณทุกคลาสกำหนดข้อมูลเฉพาะของตัวเองไว้ใน constructor แล้วส่งต่อให้คลาสแม่ผ่าน `super(...)` ตัวอย่างคลาส `BisectionMethod` กำหนดว่าอินพุตของวิธีมีอะไรบ้าง (สมการ, ช่วง XL–XU, Tolerance, Max Iterations) ข้อมูล fields นี้ถูกส่งต่อไปถึง Frontend ผ่าน API `GET /api/methods` เพื่อสร้างฟอร์มกรอกข้อมูลของแต่ละวิธีโดยอัตโนมัติ — เพิ่มวิธีใหม่ก็ได้ฟอร์มใหม่ทันทีโดยไม่ต้องแก้ Frontend

algorithms/roots.js – BisectionMethod.constructor

```
constructor() {
  super({
    key: 'bisection',
    category: 'root',
    name: 'Bisection Method',
    nameTh: 'วิธีแบ่งครึ่งช่วง',
    fields: [
      { name: 'fx', label: 'f(x) = 0', type: 'expression', placeholder: 'เช่น x^3 - x - 2' },
      { name: 'xl', label: 'XL (ปลายซ้าย)', type: 'number' },
      { name: 'xu', label: 'XU (ปลายขวา)', type: 'number' },
      { name: 'tol', label: 'Tolerance', type: 'number', default: 1e-6 },
      { name: 'maxIter', label: 'Max Iterations', type: 'int', default: 100, min: 1, max: 1000 },
    ],
  });
}
```

2.4.2 Encapsulation

ข้อมูลภายในคลาสและโมดูลถูกซ่อนไว้ ภายนอกเข้าถึงผ่านเมธอดที่กำหนดเท่านั้น `registry.js` ซ่อนโครงสร้างเก็บวัตถุ (Map) ไว้ภายใน ภายนอกเรียกผ่านฟังก์ชัน `register` / `getMethod` / `listMethods` และป้องกัน key ซ้ำตั้งแต่ตอนลงทะเบียน:

algorithms/registry.js – register

```
function register(method) {
  if (methods.has(method.meta.key)) {
    throw new Error(`method key ซ้ำ: ${method.meta.key}`);
  }
  methods.set(method.meta.key, method);
}
```

นอกจากนี้ชั้น validation ยังทำหน้าที่เป็น **ด่านเดียว** ในการตรวจอินพุตทุกชนิด (จุดทศนิยม, จำนวนเต็ม, ตารางจุด, เมทริกซ์, สมการ) โค้ดคำนวณจึงไม่ต้องตรวจข้อมูลเองซ้ำ ๆ อีก และ BaseMethod ซ่อนเงื่อนไขบังคับของ meta (ต้องมี key, category, name) ไว้ในตัว:

algorithms/BaseMethod.js – constructor

```
constructor(meta) {
  if (!meta || !meta.key || !meta.category || !meta.name) {
    throw new Error('method meta ต้องมี key, category และ name');
  }
  this.meta = meta;
}
```

2.4.3 Composition

registry.js ประกอบวัตถุวิธีคำนวณทั้ง 11 วัตถุเข้าเป็นระบบเดียว สังเกตว่าวิธีอนุพันธ์ทั้งสามตัวมาจากคลาสเดียวกันคือ DifferenceMethod แต่ถูกสร้างเป็น 3 วัตถุด้วยค่า kind ต่างกัน:

algorithms/registry.js – ลงทะเบียนวิธีคำนวณทั้ง 11 วิธี

```
const categories = new Map([
  ['root', 'Root of Equation'],
  ['linear', 'Linear Algebra Equation'],
  ['interpolation', 'Interpolation'],
  ['extrapolation', 'Extrapolation'],
  ['integration', 'Integration'],
  ['differentiation', 'Differentiation'],
]);

function register(method) {
  if (methods.has(method.meta.key)) {
    throw new Error(`method key ซ้ำ: ${method.meta.key}`);
  }
  methods.set(method.meta.key, method);
}

[
  new BisectionMethod(),
  new NewtonRaphsonMethod(),
  new GaussEliminationMethod(),
  new LinearInterpolationMethod(),
  new LagrangeInterpolationMethod(),
  new LinearExtrapolationMethod(),
  new TrapezoidalRuleMethod(),
  new SimpsonsOneThirdRuleMethod(),
  new DifferenceMethod('forward'),
  new DifferenceMethod('backward'),
  new DifferenceMethod('central'),
].forEach(register);
```

แต่ละคลาสวิธีคำนวณยัง **ประกอบ** วัตถุอื่นเข้ามาใช้ เช่น ทุกคลาสรับวัตถุฟังก์ชัน f(x) ที่ได้จาก expression parser มาใช้คำนวณโดยไม่สนใจว่า parser เขียนอย่างไรภายใน และฝั่งฐานข้อมูล history.js ประกอบวัตถุ db จาก db.js เข้ามาใช้

ตารางที่ 2.2 ความสัมพันธ์แบบ Composition ที่สำคัญ

ส่วนที่ประกอบ	ถือครอง	ความหมาย
registry.js	วัตถุวิธีคำนวณ 11 วัตถุ ใน Map	ศูนย์กลางระบบ เพิ่มวิธีใหม่ที่จุดเดียว
คลาสวิธีคำนวณทุกคลาส	ฟังก์ชัน f(x) จาก expression parser	อัลกอริทึมใช้ parser โดยไม่รู้รายละเอียดภายใน
routes/index.js	registry, history, validation	ชั้น API ประกอบทุกระบบเข้าด้วยกัน
history.js	db จาก database/db.js	ชั้นเข้าถึงข้อมูลแยกจากชั้น API
หน้าเว็บ React	MethodInput, ResultView, Dashboard	แต่ละหน้าจอเป็น Component อิสระต่อกัน

2.4.4 Polymorphism

REST API ปลายทางเดียว `POST /api/calculate` ใช้คำนวณได้ทั้ง 11 วิธี — โค้ดเรียก `method.solve(parsed)` เพียงบรรทัดเดียว แต่พฤติกรรมจริงขึ้นอยู่กับว่า method เป็นวัตถุของคลาสใด (Bisection ก็แบ่งครึ่งช่วง, Gauss ก็ตัดตัวแปร) เรียกว่า Dynamic Binding:

routes/index.js – โค้ดชุดเดียว ใช้กับวิธีใดก็ได้ (Polymorphism)

```
const method = getMethod(methodKey);
if (!method) {
  return res.status(400).json({ error: `ไม่พบวิธีคำนวณ '${methodKey}'` });
}

// แปลง/ตรวจ input ตาม field definition ของวิธีนั้น ๆ
const parsed = {};
try {
  for (const field of method.meta.fields) {
    const raw = input ? input[field.name] : undefined;
    switch (field.type) {
      case 'expression':
        parsed[field.name] = require('../validation').asExpression(raw, field.label);
        break;
      case 'number':
        parsed[field.name] = require('../validation').asFloat(raw, field.label, {
          min: field.min,
          max: field.max,
          default: field.default,
        });
        break;
      case 'int':
        parsed[field.name] = require('../validation').asInt(raw, field.label, {
          min: field.min,
          max: field.max,
          default: field.default,
        });
        break;
      case 'points':
        parsed[field.name] = require('../validation').asPoints(raw, field.label);
        break;
      case 'matrix':
        parsed[field.name] = require('../validation').asMatrix(raw, field.label);
        break;
      default:
        throw new Error(`ชนิด field ไม่รู้จัก: ${field.type}`);
    }
  }
} catch (e) {
  return res.status(e.status || 400).json({ error: e.message });
}

// คำนวณจริง
let result;
try {
  result = method.solve(parsed);
  } catch (e) {
    return res.status(e.status || 400).json({ error: e.message });
  }
```

Polymorphism ยังทำให้ผลลัพธ์มีรูปแบบเดียวกันทุกวิธี ทุก solve() คืน `{ answer, steps, table, meta }` หน้า ResultView จึงแสดงผลได้ทั้ง 11 วิธีโดยไม่ต้องมีเงื่อนไขแยกวิธี

2.4.5 Abstraction และ Inheritance

`BaseMethod` เป็นคลาสนามธรรม: ประกาศเมธอด `solve()` ไว้ให้คลาสลูกสืบทอดและเขียนทับ (override) ตามสูตรของตัวเอง ใครลืม implement โปรแกรมจะโยน error ทันที:

algorithms/BaseMethod.js – เมธอดนามธรรม

```
solve(_input) {
  throw new Error(`วิธี '${this.meta.key}' ยังไม่ได้ implement เมธอด solve()`);
}
```

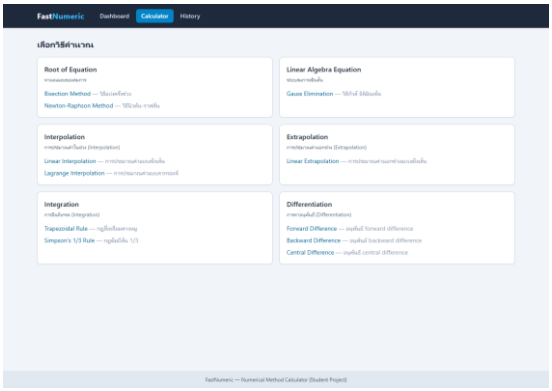
คลาสลูกทุกตัวสืบทอด (extends) จาก BaseMethod ได้คุณสมบัติร่วมโดยอัตโนมัติ ได้แก่ โครงสร้าง meta แบบเดียวกัน, การตรวจ meta ใน constructor ของคลาสแม่, การถูกเก็บใน registry และรูปแบบผลลัพธ์มาตรฐานที่ ResultView นำไปแสดงผลได้ทุกวิธี คลาสลูกเพิ่มเฉพาะคณิตศาสตร์ของตัวเอง:

การประกาศคลาสที่ใช้การสืบทอด

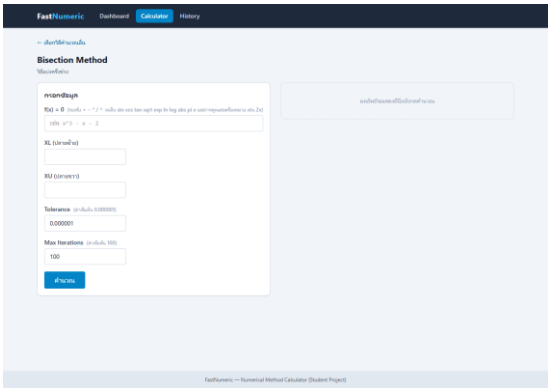
```
class BisectionMethod extends BaseMethod { ... }
class NewtonRaphsonMethod extends BaseMethod { ... }
class GaussEliminationMethod extends BaseMethod { ... }
// ... คลาสวิธีคำนวณอื่น ๆ อีก 8 คลาส extends BaseMethod เช่นเดียวกัน
```

2.5 หน้าจอโปรแกรม

หน้าจอทั้งหมดพัฒนาด้วย React + Tailwind CSS แบ่งเป็น 4 หน้าหลัก ได้แก่ Dashboard (สรุปสถิติ), Calculator (เลือกวิธี + ฟอร์มกรอก + ผลลัพธ์), History (ประวัติการคำนวณ) และรายละเอียดโจทย์ ฟอร์มกรอกอินพุตของแต่ละวิธีถูกสร้าง**อัตโนมัติ**จาก meta.fields ที่ API ส่งมา ฟิลด์แต่ละชนิด (สมการ, ตัวเลข, ตารางจุด, เมทริกซ์) มี Component ของตัวเอง เช่น MatrixEditor สำหรับกรอกเมทริกซ์ที่ขยาย/ย่อขนาดได้ และมีการตรวจอินพุตเบื้องต้นฝั่ง client ก่อนส่งไปยังเซิร์ฟเวอร์

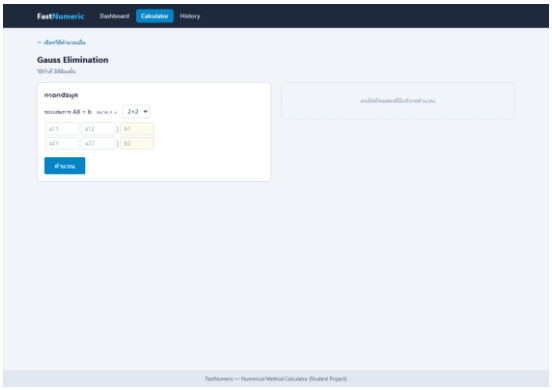


(ก) หน้าเลือกวิธีคำนวณ 6 หมวด 11 วิธี

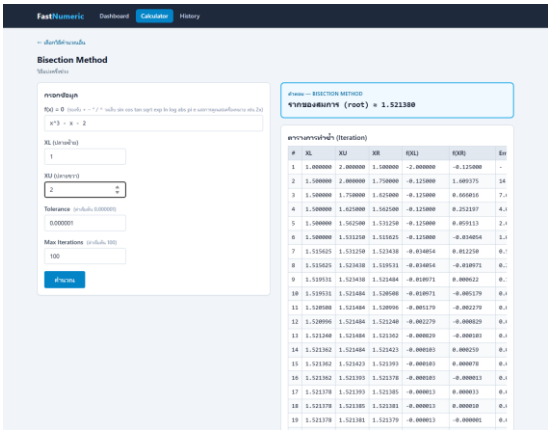


(ข) ฟอร์มอัตโนมัติของวิธี Bisection

รูปที่ 2.2 หน้าจอหลักของโปรแกรม (ภาพหน้าจอจริง)



(ก) ฟอร์มเมทริกซ์ (MatrixEditor)



(ข) ผลลัพธ์ Bisection พร้อมตารางการทำซ้ำ

รูปที่ 2.3 การกรอกอินพุตและการแสดงผลลัพธ์

ตารางที่ 2.3 องค์ประกอบของแต่ละหน้าจอ

หน้าจอ	องค์ประกอบสำคัญ
Dashboard	การ์ดสรุปจำนวนโจทย์และจำนวนครั้งที่คำนวณ, สัดส่วนการใช้งานแยกตามหมวด, รายการโจทย์ยอดนิยม 5 อันดับ (ข้อมูลทั้งหมดอ่านจาก SQLite จริง)
Calculator – เลือกวิธี	แสดงหมวดทั้ง 6 หมวดพร้อมวิธีในหมวด รายการอ่านจาก GET /api/methods โดยตรง (ไม่มีรายการแข็งในโค้ด) เพิ่มวิธีใหม่ที่เซิร์ฟเวอร์แล้วหน้าเว็บแสดงเพิ่มทันที
Calculator – ฟอร์มอินพุต	สร้างอัตโนมัติจาก meta.fields: ช่องสมการ f(x), ช่องตัวเลข, ตารางจุด (เพิ่ม/ลบแถวได้), MatrixEditor (เลือกขนาด n แล้วกรอก A และ b) พร้อมปุ่มคำนวณ
Calculator – ผลลัพธ์	แสดงคำตอบสุดท้าย, ขั้นตอนวิธีทำทีละส่วน (สูตร, การทำซ้ำ, การแทนค่า), ตารางการทำซ้ำเต็ม และป้ายเตือนกรณีคำตอบเป็นค่าประมาณ
History	รายการโจทย์ที่เคยคำนวณ (หมวด วิธี อินพุต จำนวนครั้ง เวลาล่าสุด) เปิดดูรายละเอียดและลบรายการได้

2.6 อธิบาย REST API และการจัดการเหตุการณ์

การสื่อสารระหว่าง Frontend กับ Backend ใช้ REST API ภายใต้ `/api` ทั้งหมด เมื่อผู้ใช้กดปุ่ม **คำนวณ** หน้าเว็บส่ง `POST /api/calculate` พร้อม `{ method, input }` ฝั่งเซิร์ฟเวอร์ทำงานตามลำดับ: (1) ค้นหาวัตถุวิธีคำนวณจาก registry, (2) ตรวจและแปลงอินพุตทีละฟิลด์ตาม field.type ที่ด่าน validation, (3) เรียก `method.solve(parsed)`, (4) บันทึกลงฐานข้อมูล และ (5) ส่งผลลัพธ์กลับ — ถ้าขั้นใดผิดพลาดจะตอบ HTTP 400 พร้อมข้อความไทยอธิบายสาเหตุ

ตารางที่ 2.4 ปลายทาง REST API และหน้าที่

ปลายทาง	เมธอด	หน้าที่
/api/methods	GET	รายการหมวดและวิธีคำนวณพร้อม meta.fields (ใช้สร้างฟอร์มอัตโนมัติ)
/api/calculate	POST	ตรวจอินพุต คำนวณด้วย method.solve() แล้วบันทึกประวัติลง SQLite
/api/history	GET	รายการโจทย์ล่าสุด พร้อมจำนวนครั้งที่คำนวณของแต่ละโจทย์ (รองรับ limit / offset)
/api/history/:id	GET	รายละเอียดโจทย์ ประวัติการคำนวณทุกครั้ง และผลลัพธ์ล่าสุด
/api/history/:id	DELETE	ลบโจทย์และประวัติการคำนวณทั้งหมดของโจทย์นั้น
/api/stats	GET	สถิติสำหรับ Dashboard: จำนวนโจทย์, จำนวนครั้ง, แยกตามหมวด, โจทย์ยอดนิยม

ฝั่งหน้าเว็บจัดการเหตุการณ์ด้วย React hooks เป็นหลัก การเปลี่ยนค่าในฟอร์ม (onChange) อัปเดต state ของ Component ทันทีและตรวจความถูกต้องเบื้องต้น การกดปุ่มคำนวณ (onSubmit) ส่ง request แล้วแสดงสถานะกำลังโหลดจนได้ผลตอบ กรณีเซิร์ฟเวอร์ตอบ error หน้าเว็บแสดงกล่องข้อความไทยที่ได้จาก API ให้ผู้ใช้แก้อินพุตแล้วส่งใหม่ได้ทันที นอกจากนี้หน้าเว็บยังตรวจว่า response จาก API มีโครงสร้างครบก่อนนำไปแสดงผล เพื่อป้องกันหน้าเว็บพังเมื่อเซิร์ฟเวอร์ยังไม่พร้อม

2.7 อธิบายอัลกอริทึมที่สำคัญในโปรแกรม

2.7.1 Expression Parser (แยกวิเคราะห์สมการโดยไม่ใช้ eval)

เพื่อความปลอดภัยและเพื่อให้ข้อความ error อ่านเข้าใจได้ โปรแกรมแยกวิเคราะห์สมการ f(x) จากสตริงด้วย Tokenizer + Recursive Descent Parser ที่เขียนขึ้นเอง รองรับตัวดำเนินการ + − * / ^ วงเล็บ ฟังก์ชัน

sin cos tan sqrt exp ln log abs ค่าคงที่ pi, e และการคูณละเครื่องหมาย (เช่น 2x, 3(x+1)) Tokenizer แปลงสตริงเป็นชุดโทเคนก่อน ถ้าพบอักขระที่ไม่รู้จักจะโยน ExprError พร้อมตำแหน่งที่ผิด:

expression/parser.js – tokenize

```
function tokenize(src) {
  const tokens = [];
  let i = 0;
  while (i < src.length) {
    const ch = src[i];
    if (ch === ' ' || ch === '\t' || ch === '\n' || ch === '\r') {
      i++;
      continue;
    }
    if (/[0-9.]/.test(ch)) {
      const m = /^(?:\d+\.?\d*|\.\d+)(?:[eE][+-]?\d+)?/.exec(src.slice(i));
      if (!m) throw new ExprError(`ตัวเลขไม่ถูกต้องที่ตำแหน่งที่ ${i + 1}`);
      tokens.push({ type: 'num', value: parseFloat(m[0]) });
      i += m[0].length;
      continue;
    }
    if (/[a-zA-Z_]/.test(ch)) {
      const m = /^[a-zA-Z_][a-zA-Z_0-9]*/.exec(src.slice(i));
      tokens.push({ type: 'ident', value: m[0] });
      i += m[0].length;
      continue;
    }
    if ('+-*/^(),'.includes(ch)) {
      tokens.push({ type: ch });
      i++;
      continue;
    }
    throw new ExprError(`ไม่รู้จักอักขระ '${ch}' ที่ตำแหน่งที่ ${i + 1}`);
  }
  return tokens;
}
```

จากนั้นคลาส `Parser` ใช้หลัก Recursive Descent แบ่งสมการตามลำดับความสำคัญของตัวดำเนินการ (expr → term → power → unary → atom) ให้ได้ AST แล้วเดินต้นไม้เพื่อประเมินค่าที่ x ใด ๆ ได้

2.7.2 Bisection Method — การทำซ้ำแบบแบ่งครึ่งช่วง

ก่อนทำซ้ำโปรแกรมตรวจเงื่อนไขบังคับก่อน ได้แก่ XL ต้องน้อยกว่า XU และ f(XL)·f(XU) ต้องมีเครื่องหมายต่างกัน ถ้าเครื่องหมายเดียวกันจะแจ้งว่า **ไม่มีการเปลี่ยนเครื่องหมายในช่วงนี้ จึงใช้ Bisection ไม่ได้ (ลองเปลี่ยนช่วง)** แกนของการทำซ้ำคือหาจุดกึ่งกลาง XR = (XL + XU) / 2 แล้วเลือกครึ่งที่มีการเปลี่ยนเครื่องหมายไว้ต่อ วัดความคลาดเคลื่อนด้วย |(XR ใหม่ − XR เดิม) / XR ใหม่| × 100 จนน้อยกว่า Tolerance:

algorithms/roots.js – BisectionMethod.solve (ส่วนการทำซ้ำ)

```
const rows = [];
let a = xl;
let b = xu;
let prev = null;
let iter = 0;
let root = (a + b) / 2;


for (; iter < maxIter; iter++) {
  const fa = iter === 0 ? fxl : evalAt(fx, a, 'f(XL)');
  root = (a + b) / 2;
  const froot = evalAt(fx, root, 'f(x)');
  const error = prev === null ? '-' : Math.abs((root - prev) / root) * 100;
  rows.push({
    iter: iter + 1,
    xl: a,
    xu: b,
    xr: root,
```

```
    fxl: fa,
    fxr: froot,
    error,
  });
  if (froot === 0) break;
  if (fa * froot < 0) {
    b = root;
  } else {
    a = root;
  }
  if (prev !== null && Math.abs((root - prev) / root) * 100 < tol) break;
  prev = root;
```

ทุกรอบการทำซ้ำถูกเก็บลงตาราง (XL, XU, XR, f(XL), f(XR), Error %)
เพื่อแสดงให้ผู้ใช้เห็นการลู่เข้าของคำตอบทีละรอบ

2.7.3 Gauss Elimination — การตัดตัวแปรแบบ Partial Pivoting

แก้ระบบสมการ AX = b ด้วย Forward Elimination
โดยทุกคอลัมน์จะเลือกแถวที่มีค่าสัมบูรณ์มากที่สุดเป็น pivot ก่อน (Partial Pivoting) เพื่อลดปัญหา floating point error และตรวจว่า pivot ไม่ใกล้ศูนย์ (เมทริกซ์เอกฐน) จากนั้นตัดตัวแปรแถวล่างด้วย $R_i \leftarrow R_i - m \cdot R_{pivot}$ แล้วปิดท้ายด้วยการแทนค่าย้อนกลับ (Back Substitution)
ทุกขั้นตอนถูกบันทึกเป็นเมทริกซ์ข้อความเพื่อแสดงวิธีทำให้ผู้ใช้:

algorithms/linear.js – Forward Elimination พร้อม Partial Pivoting

```
// Forward Elimination
for (let k = 0; k < n - 1; k++) {
  // Partial Pivoting: สลับแถวให้ pivot มีค่าสัมบูรณ์มากที่สุด (ลด floating point error)
  let maxRow = k;
  for (let i = k + 1; i < n; i++) {
    if (Math.abs(A[i][k]) > Math.abs(A[maxRow][k])) maxRow = i;
  }
  if (Math.abs(A[maxRow][k]) < 1e-12) {
    throw badRequest(
      `เมทริกซ์เป็น Singular (ไม่มีคำตอบหรือมีไม่จำกัด) — ไม่สามารถหา pivot ที่คอลัมน์ ${k + 1}`
    );
  }
  if (maxRow !== k) {
    [A[k], A[maxRow]] = [A[maxRow], A[k]];
    [b[k], b[maxRow]] = [b[maxRow], b[k]];
    steps.push({
      title: `แถวที่ ${k + 1}: สลับแถว (Partial Pivoting) R${k + 1} ↔ R${maxRow + 1}`,
      lines: [this._matrixString(A, b, n)],
    });
  }


  for (let i = k + 1; i < n; i++) {
    const m = A[i][k] / A[k][k];
    for (let j = k; j < n; j++) {
      A[i][j] -= m * A[k][j];
    }
    b[i] -= m * b[k];
    A[i][k] = 0; // บังคับเป็นศูนย์เพื่อไม่ให้เศษ floating point ค้าง
      }
```

algorithms/linear.js – Back Substitution

```
// Back Substitution
const x = new Array(n).fill(0);
for (let i = n - 1; i >= 0; i--) {
  let s = b[i];
  for (let j = i + 1; j < n; j++) {
    s -= A[i][j] * x[j];
  }
  x[i] = s / A[i][i];
    }
```

2.7.4 Newton-Raphson — อนุพันธ์เชิงตัวเลข

สูตร Newton-Raphson ต้องการอนุพันธ์ f'(x) โปรแกรมคำนวณด้วย central difference ความแม่นยำสูง โดยให้ h แปรผันตามขนาดของ x และตรวจกรณี f'(x) = 0 เพื่อแจ้งผู้ใช้ให้เปลี่ยนค่าเริ่มต้น X0 แทนการพัง:

algorithms/roots.js – อนุพันธ์เชิงตัวเลข

```
const df = (x) => {
  const h = Math.max(1e-9, Math.abs(x) * 1e-8);
  return (evalAt(fx, x + h, "f(x+h)") - evalAt(fx, x - h, "f(x-h)")) / (2 * h);
```

2.7.5 การนับโจทย์ซ้ำด้วย SHA-256 Hash

เพื่อให้โจทย์เดิมไม่ถูกบันทึกซ้ำในตารางประวัติ โปรแกรมสร้าง hash จาก (หมวด + วิธี + อินพุต) โดยเรียง key ของอินพุตก่อนเสมอ เพื่อให้ {a, b} กับ {b, a} ให้ hash เดียวกัน hash นี้ใช้เป็น UNIQUE constraint ของตาราง problem โจทย์เดิมจะเพิ่มเพียง **จำนวนครั้งที่คำนวณ**:

database/history.js – hashInput

```
function hashInput(category, method, input) {
  // เรียง key ก่อน hash ให้ {a,b} กับ {b,a} ให้ hash เดียวกัน
  const canonical = JSON.stringify(input, (_k, v) =>
    v && typeof v === 'object' && !Array.isArray(v)
      ? Object.keys(v).sort().reduce((acc, key) => ({ ...acc, [key]: v[key] }), {})
      : v
  );
  return crypto.createHash('sha256').update(`${category}|${method}|${canonical}`).digest('hex');
}
```

บทที่ 3
สรุปผลและข้อเสนอแนะ

โครงงาน FastNumeric พัฒนาเว็บแอปพลิเคชันคำนวณวิธีเชิงตัวเลขได้สำเร็จตามขอบเขตหลัก ครอบคลุม 6 หมวด 11 วิธีครบถ้วน แสดงขั้นตอนการคำนวณและตารางการทำซ้ำทุกวิธี มีระบบประวัติการคำนวณที่นับโจทย์ซ้ำ และ Dashboard สรุปสถิติ โครงสร้างโปรแกรมแบ่งชั้นชัดเจนตามหลักการเขียนโปรแกรมเชิงวัตถุทุกหลักการ และเพิ่มวิธีคำนวณใหม่ได้โดยแก้โค้ดเพียง 2 จุด (คลาสใหม่ + บรรทัด register)

3.1 ปัญหาที่พบระหว่างการพัฒนา

ตารางที่ 3.1 ปัญหาที่พบระหว่างพัฒนาและแนวทางแก้ไข

ปัญหา	สาเหตุ	แนวทางแก้ไข (หลักฐาน)
การใช้ eval() กับสมการ f(x) เสี่ยงต่อความปลอดภัยและข้อความ error อ่านไม่เข้าใจ	eval ประเมินโค้ด JavaScript ใด ๆ ได้ และ SyntaxError ของเอนจินไม่ได้เล่าว่าผู้ใช้กรอกผิดตรงไหน	เขียน Expression Parser เอง (Tokenizer + Recursive Descent) แปลง error เป็นข้อความไทยระบุตำแหน่งอักขระที่ผิด เช่น 'ไม่รู้จักอักขระ @ ที่ตำแหน่งที่ 3' (expression/parser.js)
หน้าเว็บแก้ระบบสมการด้วย Gauss ไม่ได้ ขึ้น 'รูปแบบข้อมูลเมทริกซ์ไม่ถูกต้อง'	Frontend ส่ง key เมทริกซ์เป็น A (พิมพ์ใหญ่) แต่ asMatrix ฝั่งเซิร์ฟเวอร์รับเฉพาะ a (พิมพ์เล็ก)	ปรับ asMatrix ให้รับทั้ง m.a และ m.A (validation/index.js) ทดสอบผ่านหน้าเว็บและ API จริง
หน้าเว็บขึ้น 'Cannot read properties of null' เมื่อ API ยังไม่ตอบ	โค้ดฝั่ง client เข้าถึงข้อมูล response ก่อนที่ request จะเสร็จหรือเมื่อ response ไม่มีโครงสร้างที่คาดไว้	ตั้ง Vite dev proxy ให้ /api ไปยังเซิร์ฟเวอร์ Express และตรวจ response ก่อน setState (เช่น Array.isArray(d.categories))
วิธี Simpson ต้องการจำนวนช่วงเป็นเลขคู่ แต่ผู้ใช้กรอกเลขคี่ได้	อัลกอริทึม Simpson's 1/3 ใช้สูตรกับช่วงคู่เท่านั้น ถ้ารับเลขคี่แล้วผลลัพธ์จะผิดโดยไม่รู้ตัว	ตรวจที่ด่าน validation แล้วแจ้งข้อความไทยพร้อมคำแนะนำให้กรอกใหม่ (validation/index.js)
Newton-Raphson ไม่ลู่เข้าเมื่อ f'(x) = 0 หรือเริ่มที่จุดที่ไม่เหมาะสม	สูตรต้องหารด้วย f'(x) ซึ่งเป็นศูนย์ได้ และการลู่เข้าขึ้นกับจุดเริ่มต้น	ตรวจ d0 === 0 แล้วแจ้ง 'ลองเปลี่ยนค่าเริ่มต้น X0' กรณีครบรอบสูงสุดแสดงป้ายเตือนว่าคำตอบเป็นค่าประมาณ (roots.js)
โจทย์เดิมถูกบันทึกซ้ำ ๆ ทำให้ตารางประวัติรก	การบันทึกทุกครั้งไม่แยกว่าเป็นโจทย์เดิมหรือโจทย์ใหม่	ใช้ SHA-256 hash ของ (หมวด + วิธี + อินพุตที่เรียง key แล้ว) เป็น UNIQUE constraint โจทย์เดิมจะเพิ่มแค่จำนวนครั้งที่คำนวณ (history.js)

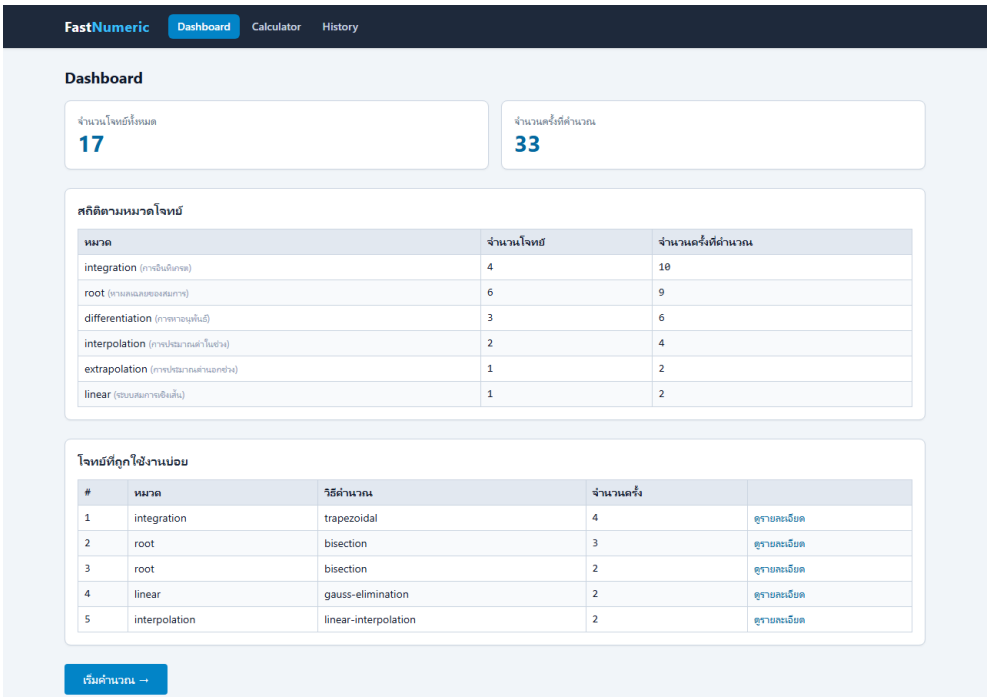
หมายเหตุ: ปัญหาข้างต้นสรุปจากประสบการณ์การพัฒนาจริงและสามารถชี้ตำแหน่งโค้ดที่เกี่ยวข้องได้ทุกข้อ โดยเฉพาะปัญหาเมทริกซ์ A/a เป็นบั๊กระหว่าง Frontend กับ Backend ที่พบและแก้ไขจริงระหว่างการทดสอบ

3.2 จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

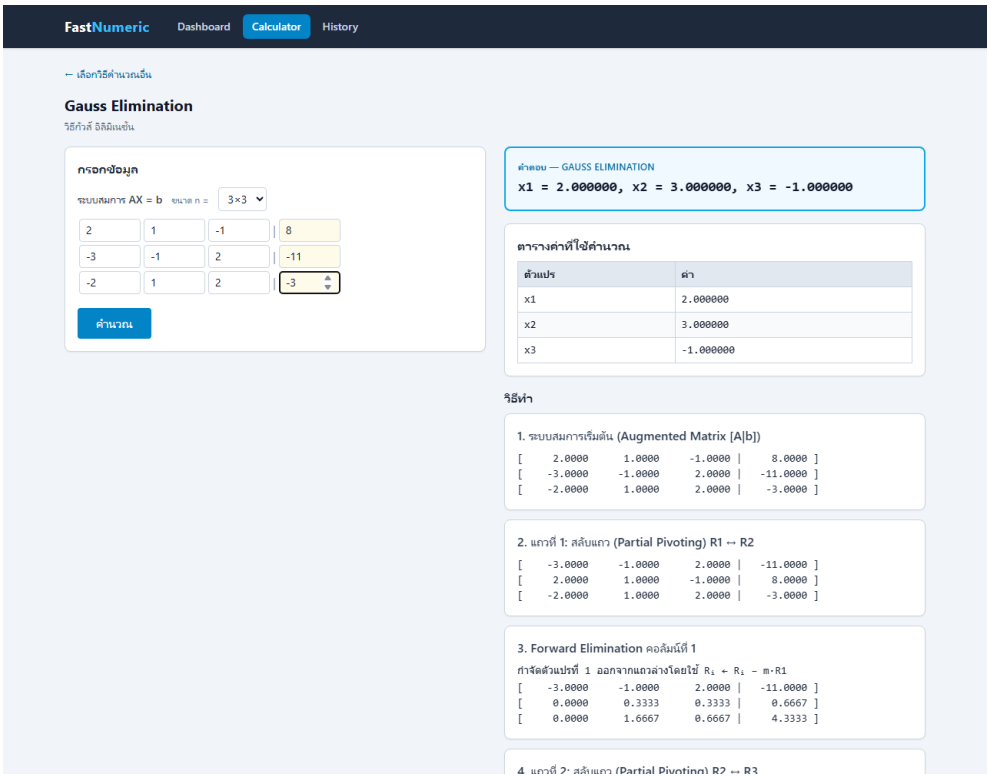
1. **แสดงขั้นตอนการคำนวณครบทุกวิธี** ไม่ใช่แค่คำตอบ — เช่น Gauss Elimination แสดง Augmented Matrix ตั้งต้น การสลับแถวแบบ Partial Pivoting การตัดตัวแปรทีละคอลัมน์ และการแทนค่าย้อนกลับให้เห็นทุกขั้น

2. **ฟอร์มอินพุตสร้างอัตโนมัติจากข้อมูล (Data-driven UI)** นิยามฟิลด์ของแต่ละวิธีอยู่ที่ meta.fields ของคลาส หน้าเว็บอ่านจาก API แล้วสร้างฟอร์มเอง เพิ่มวิธีใหม่ที่เซิร์ฟเวอร์ก็มีหน้าใช้งานทันทีโดยไม่แก้ Frontend
3. **Expression Parser เขียนเองโดยไม่ใช้ eval()** ปลอดภัยและให้ข้อความ error ภาษาไทยระบุตำแหน่งอักขระที่ผิด รองรับฟังก์ชันคณิตศาสตร์ ค่าคงที่ และการคูณละเครื่องหมาย
4. **จัดการข้อผิดพลาดเป็นภาษาไทยที่บอกทางแก้** ทุกกรณีผิดพลาด (ช่วงไม่เปลี่ยนเครื่องหมาย, เมทริกซ์เอกฐน, จำนวนช่วงเลขคี่, f'(x) = 0) มีข้อความอธิบายสาเหตุและคำแนะนำ ไม่ใช่ stack trace
5. **ประวัติการคำนวณที่นับโจทย์ซ้ำอัตโนมัติ** ใช้ SHA-256 hash แยกโจทย์เดิม/ใหม่ ผู้ใช้เห็นว่าโจทย์ไหนคำนวณบ่อย และ Dashboard สรุปสถิติจากข้อมูลจริง
6. **ติดตั้งได้ด้วยคำสั่งเดียว** Docker Compose build ทั้งหน้าเว็บและเซิร์ฟเวอร์ พร้อม health check และ named volume ทำให้ข้อมูลไม่หายเมื่อรีสตาร์ทคอนเทนเนอร์

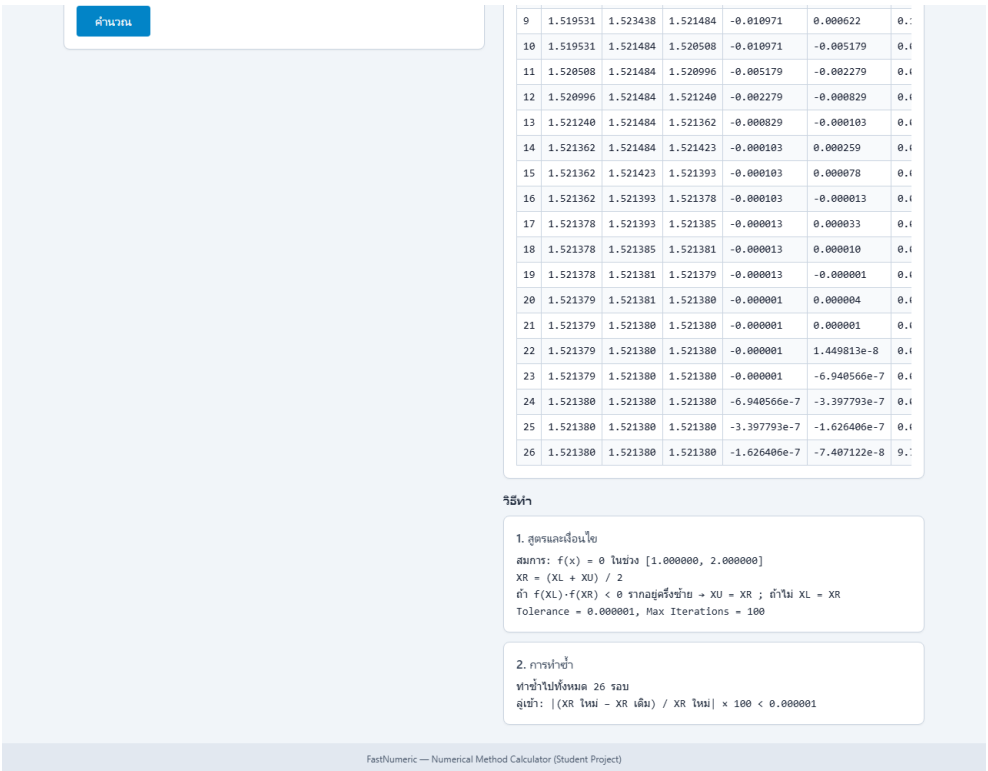
รูปที่ 3.1 ถึง 3.4 เป็นภาพหน้าจอจริงของโปรแกรมที่แสดงจุดเด่นดังกล่าว



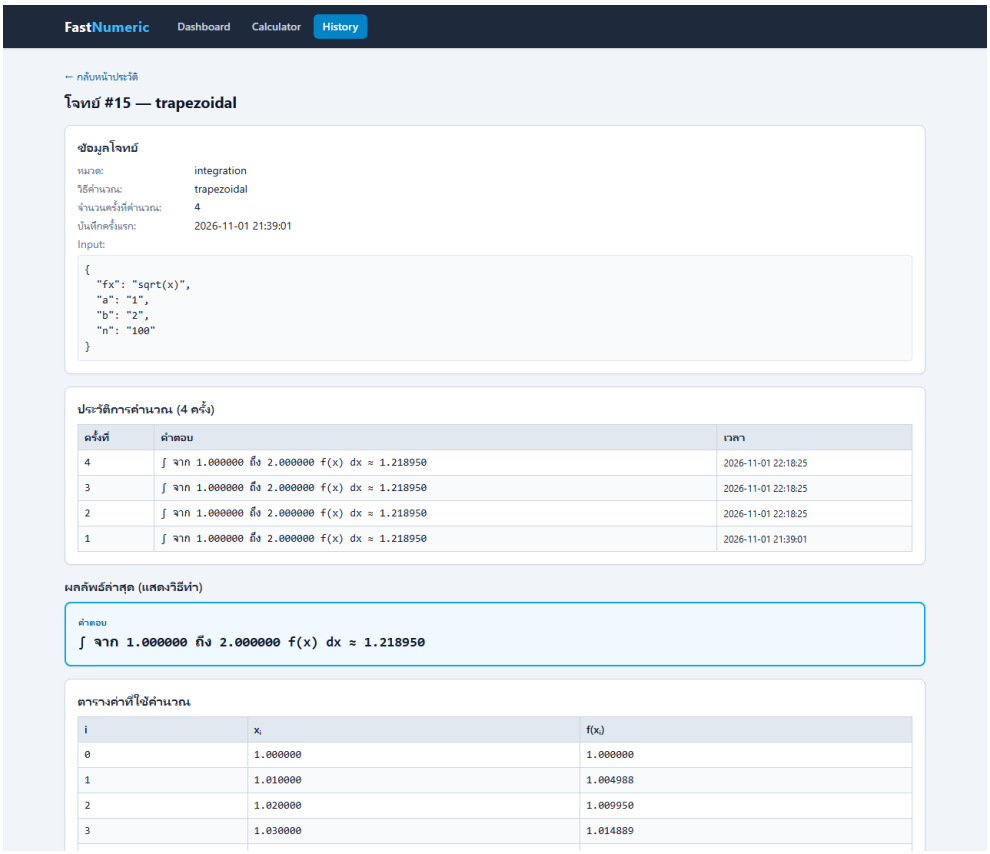
รูปที่ 3.1 หน้า Dashboard — สถิติการใช้งานและโจทย์ยอดนิยม อ่านจาก SQLite จริง



รูปที่ 3.2 ผลลัพธ์ Gauss Elimination — คำตอบ x1=2, x2=3, x3=−1 พร้อมวิธีทำทีละขั้น



รูปที่ 3.3 ตารางการทำซ้ำ Bisection ลู่เข้าหาราก ≈ 1.521380 พร้อม Error (%) ที่ลดลงทุกรอบ



รูปที่ 3.4 รายละเอียดโจทย์ — แสดงจำนวนครั้งที่คำนวณ อินพุตเดิม และผลลัพธ์ล่าสุด

3.3 การทดสอบระบบ

ระบบผ่านการทดสอบดังนี้: (1) ทดสอบอัลกอริทึมทั้ง 11 วิธีด้วยโจทย์ที่รู้คำตอบจริง เช่น Bisection และ Newton-Raphson ของ $x^3 - x - 2 = 0$ ได้ราก ≈ 1.521380, Gauss ระบบ 3×3 ได้ x = (2, 3, −1), Simpson ของ x^2 บน [0, 1] ได้ 1/3 แม่นถึงหลักทศนิยมที่คาดหมาย (2) ทดสอบกรณี error (หารด้วยศูนย์, เมทริกซ์เอกฐน, f(XL)·f(XU) > 0, x ซ้ำในตารางจุด, จำนวนช่วงเลขคี่ ฯลฯ)
ระบบแจ้งข้อความไทยถูกต้องทุกกรณีและไม่ทำให้เซิร์ฟเวอร์ล้ม (3) ทดสอบผ่านหน้าเว็บจริงครบทุกวิธี รวมถึงกรณีเมทริกซ์ที่เคยพบบั๊ก A/a และ (4)
ทดสอบการนับโจทย์ซ้ำและการคงอยู่ของข้อมูลหลังรีสตาร์ทเซิร์ฟเวอร์ (ข้อมูลเก็บในไฟล์ SQLite บน volume) ผ่านครบ

3.4 คำแนะนำสำหรับการพัฒนาต่อ

1. **เพิ่มการวาดกราฟ f(x)** พร้อมประมาณการตำแหน่งราก เพื่อช่วยผู้ใช้เลือกช่วง XL–XU ที่เหมาะสมก่อนคำนวณ
2. **เพิ่มวิธีคำนวณขั้นสูง** เช่น Secant Method, Gauss-Jordan, Newton's Divided Difference Interpolation ซึ่งทำได้ง่ายจากโครงสร้างเดิม: สร้างคลาสใหม่ extends BaseMethod แล้วเพิ่มบรรทัด register ใน registry.js
3. **เพิ่มการล็อกอินผู้ใช้** เพื่อแยกประวัติการคำนวณของแต่ละคนและเก็บสถิติรายบุคคล
4. **ส่งออกผลลัพธ์เป็น PDF หรือรูปภาพ** เพื่อให้ผู้ใช้แนบขั้นตอนการคำนวณลงรายงานของตนเองได้โดยตรง

5. **เพิ่มชุดทดสอบอัตโนมัติ** (เช่น Vitest / Supertest) ครอบคลุมทั้งอัลกอริทึมและ API เพื่อให้มั่นใจว่าการแก้โค้ดในอนาคตไม่ทำให้วิธีเดิมเสีย